S.G.Ganesh

Understanding Bit-fields in C

One important feature that distinguishes C as a systems programming language is its support for bit-fields. Let us explore this feature in this column.

In C, structure members can be specified with size in number of bits, and this feature is known as bit-fields. Bit-fields are important for low-level (i.e., for systems programming) tasks such as directly accessing systems resources, processing, reading and writing in terms of streams of bits (such as processing packets in network programming), cryptography (encoding or decoding data with complex bit-manipulation), etc.

Consider the example of reading the components of a floating-point number. A 4-byte floating-point number in the IEEE 754 standard consists of the following:

- The first bit is reserved for the sign bit—it is 1 if the number is negative and 0 if it is positive.
- The next 8 bits are used to store the exponent in the unsigned form. When treated as a signed exponent, this exponent value ranges from -127 to +128. When treated as an unsigned value, its value ranges from 0 to 255.
- The remaining 23 bits are used to store the mantissa.

Here is a program to print the value of a floating-point number into its constituents:

```
struct FP {
// the order of the members depends on the
// endian scheme of the underlying machine
      unsigned int mantissa : 23;
     unsigned int exponent : 8;
      unsigned int sign : 1;
} *fp;

int main() {
       float f = -1.0f;
       fp = (struct FP *)&f;

printf(" sign = %s, biased exponent = %u,
mantissa = %u ", fp->sign ? "negative" : "positive",
fp->exponent, fp->mantissa);
}
```

For the floating-point number -1.0, this program prints:

```
sign = negative, biased exponent = 127, mantissa = 0
```

Since the sign of the floating-point number is negative, the value of the sign bit is 1. Since the exponent is actual 0, in unsigned exponent format, it is represented as 127, and hence that value is printed. The mantissa in this case is 0, and hence it is printed as it is. To understand how floating-point arithmetic works, see the Wikipedia article *http://en.wikipedia.org/wiki/Single_precision*.

An alternative to using bit-fields is to use integers directly, and manipulate them using bitwise operators (such as &, |, ~, etc). In the case of reading the components of a floating-point number, we could use bitwise operations also. However, in many cases, such manipulation is a round-about way to achieve what we need, and the solution using bit-fields provides a more direct solution and hence is a useful feature.

There are numerous limitations in using bit-fields. For example, you cannot apply operators such as *& (addressof), sizeof* to bit-fields. This is because these operators operate in terms of bytes (not bits) and the bit-fields operate in terms of bits (not bytes), so you cannot use these operators. In other words, an expression such as *sizeof(fp->sign)* will result in a compiler error. Another reason is that the underlying machine supports addressing in terms of bytes, and not bits, and hence such operators are not feasible. Then how does it work when expressions such as *fp->sign*, or *fp->exponent* are used in this program? Note that C allows only integral types as bit-fields, and hence expressions referring to the bit-fields are converted to integers. In this program, as you can observe, we used the %u format specifier, which is for an unsigned integer—the bit-field value was converted into an integer and that is why the program worked.

Those new to bit-fields face numerous surprises when they try using them. This is because a lot of low-level details come into the picture while using them. In the programming example for bit-fields, you might have noticed the reversal in the order of the sign, exponent and mantissa, which is because of the underlying *endian* scheme followed. Endian refers to how bytes are stored in memory (see the Wikipedia article *http://en.wikipedia.org/wiki/Endianness* for more details).

Can you explain the following simple program that makes use of a bit-field?

```
struct bitfield {
    int bit : 1;
} BIT;
int main() {
   BIT.bit = 1;
   printf(" sizeof BIT is = %d\n", sizeof(BIT));
   printf(" value of bit is = %d ", BIT.bit);
```